4-Bit Adder in Qiskit

Abert Kabore

PhD student in Advanced and Equitable Computing

This circuit extends the SUM, CARRY, and reverse-CARRY design in QC_Ekert_FA1bit_and_2bit_examp.ipynb from two bits to four bits. The initial carry is zero. Five measured bits represent the sum, including the final carry-out.

4-Bit Adder Code

```
from qiskit import QuantumCircuit
from qiskit_aer.primitives import SamplerV2
from qiskit.visualization import plot_histogram

# Define functions for the ripple carry adder.
def sum(circ, cin, a, b):
    circ.cx(a,b)
    circ.cx(cin,b)

def carry(circ, cin, a, b, cout):
    circ.ccx(a, b, cout)
    circ.cx(a, b)
    circ.ccx(cin, b, cout)

def carry_rev(circ, cin, a, b, cout):
    circ.ccx(cin, b, cout)
    circ.cx(a, b)
    circ.ccx(a, b, cout)

from qiskit import QuantumRegister, ClassicalRegister


def adder_qubits(i):
    """Use the (carry, a, b) layout from the supplied two-bit circuit."""
    return 3 * i, 3 * i + 1, 3 * i + 2


def ripple_carry_adder(a, b):
    """Add two unsigned four-bit integers with initial carry zero."""
    if not (0 <= a < 16 and 0 <= b < 16):
        raise ValueError("a and b must be integers from 0 to 15")

    qr = QuantumRegister(13, "q")
    cr = ClassicalRegister(5, "s")
    qc = QuantumCircuit(qr, cr)

    # Load a_i and b_i into the same layout as the two-bit circuit.
    for i in range(4):
        _, ai, bi = adder_qubits(i)
        if (a >> i) & 1:
            qc.x(ai)
        if (b >> i) & 1:
            qc.x(bi)
    qc.barrier()
```

4-Bit Adder Code (continued)

```
    # Compute the four carries.
    for i in range(4):
        ci, ai, bi = adder_qubits(i)
        carry(qc, ci, ai, bi, ci + 3)
    qc.barrier()

    # Compute the most significant sum bit.
    ci, ai, bi = adder_qubits(3)
    qc.cx(ai, bi)
    sum(qc, ci, ai, bi)
    qc.barrier()

    # Reverse each remaining carry and compute its sum bit.
    for i in range(2, -1, -1):
        ci, ai, bi = adder_qubits(i)
        carry_rev(qc, ci, ai, bi, ci + 3)
        sum(qc, ci, ai, bi)
        qc.barrier()

    for i in range(4):
        qc.measure(adder_qubits(i)[2], cr[i])
    qc.measure(12, cr[4])
    return qc


def run_counts(circuits, shots=1024):
    """Return measured counts from the local Qiskit Aer simulator."""
    result = SamplerV2(seed=4).run(circuits, shots=shots).result()
    return [r.data.s.get_counts() for r in result]
```

Test Cases

The five tests use four-bit binary operands and 1,024 shots per circuit on Qiskit Aer. Each assertion compares the measured counts with the expected five-bit sum.

```
test_cases = [
    (0b0000, 0b0000),
    (0b0101, 0b0010),
    (0b0111, 0b0001),
    (0b1001, 0b0111),
    (0b1111, 0b1111),
]

circuits = [ripple_carry_adder(a, b) for a, b in test_cases]
all_counts = run_counts(circuits)

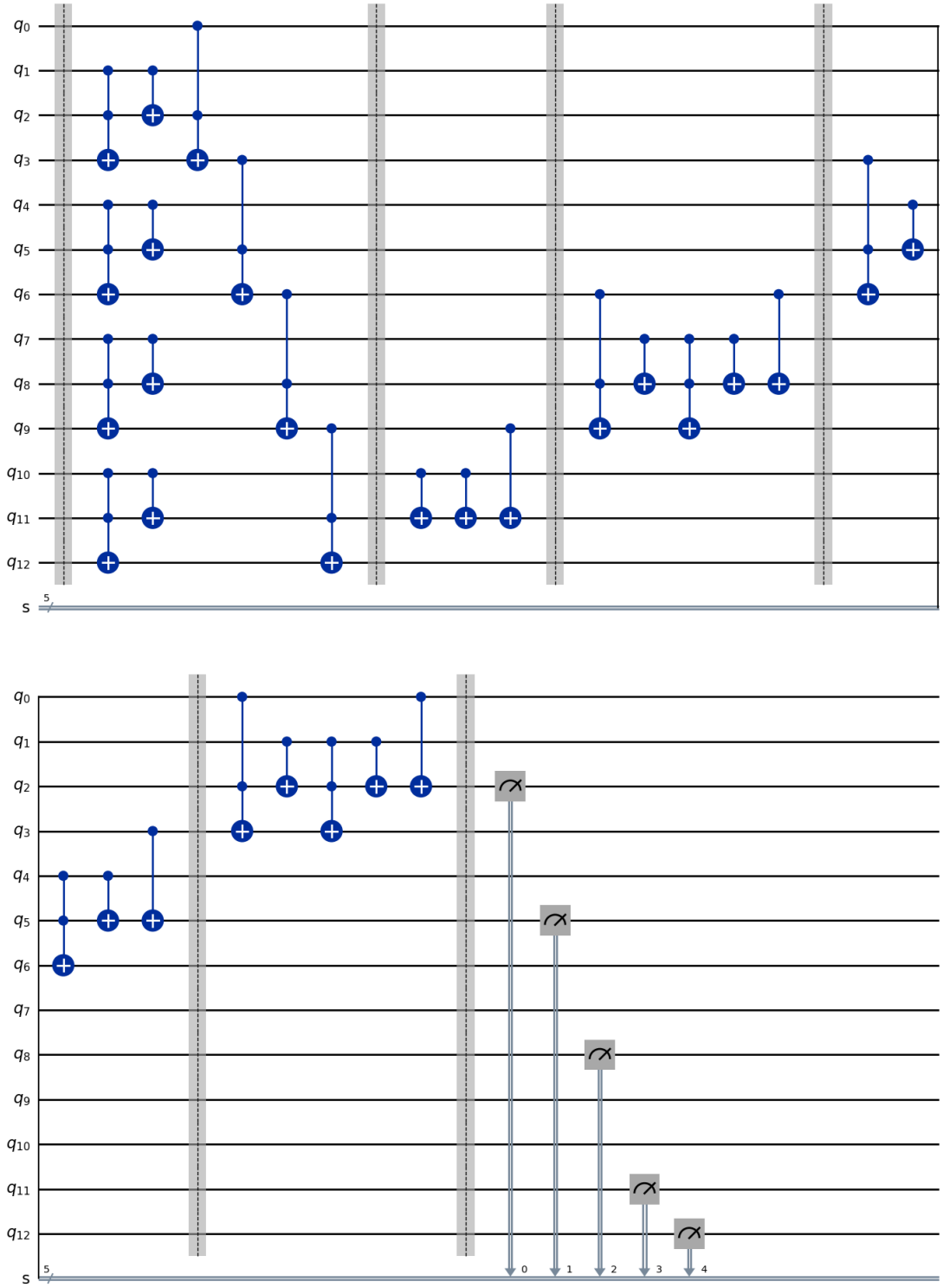
for index, ((a, b), counts) in enumerate(zip(test_cases, all_counts), 1):
    expected = f"{a + b:05b}"
    assert counts == {expected: 1024}, counts
    print(
        f"Test {index}: {a:04b} + {b:04b} = {expected}; "
        f"counts = {counts}; PASS"
    )
```

Test Results

```
Test 1: 0000 + 0000 = 00000; counts = {'00000': 1024}; PASS
Test 2: 0101 + 0010 = 00111; counts = {'00111': 1024}; PASS
Test 3: 0111 + 0001 = 01000; counts = {'01000': 1024}; PASS
Test 4: 1001 + 0111 = 10000; counts = {'10000': 1024}; PASS
Test 5: 1111 + 1111 = 11110; counts = {'11110': 1024}; PASS
```

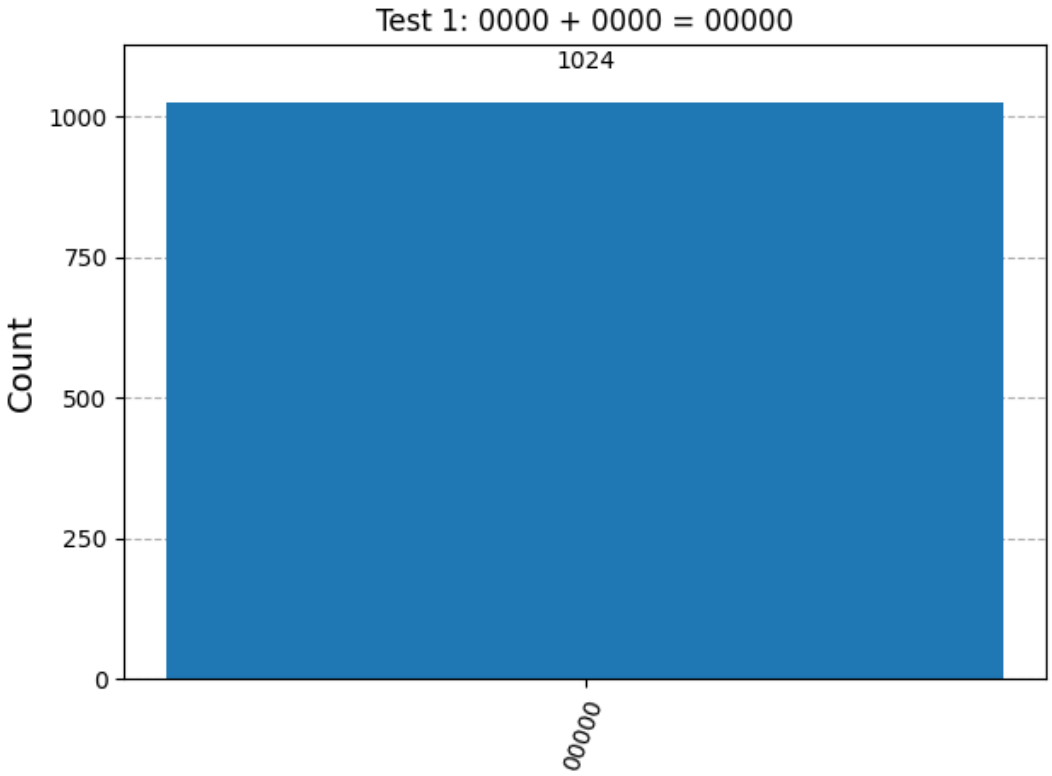
Test 1: 0000 + 0000 = 00000 (Circuit)

```
circuits[0].draw(output="mpl", fold=20)
```



Test 1: 0000 + 0000 = 00000

```
a, b = test_cases[0]
counts = all_counts[0]
title = f"Test 1: {a:04b} + {b:04b} = {a + b:05b}"
plot_histogram(counts, title=title)
```

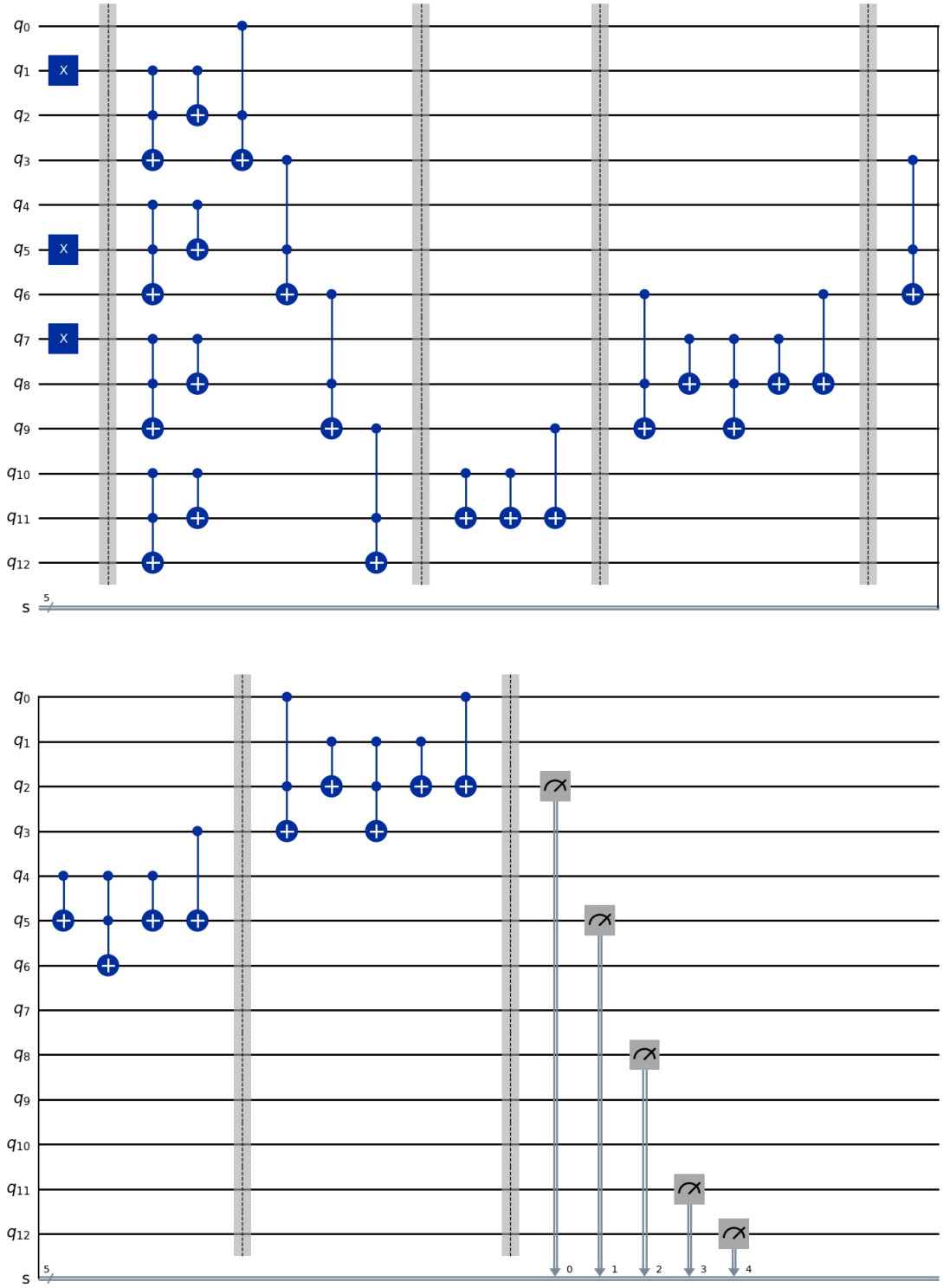


Observations

- The output 00000 occurred in all 1,024 shots, matching 0000 + 0000 = 00000.
- The four sum bits and the final carry-out are all zero.

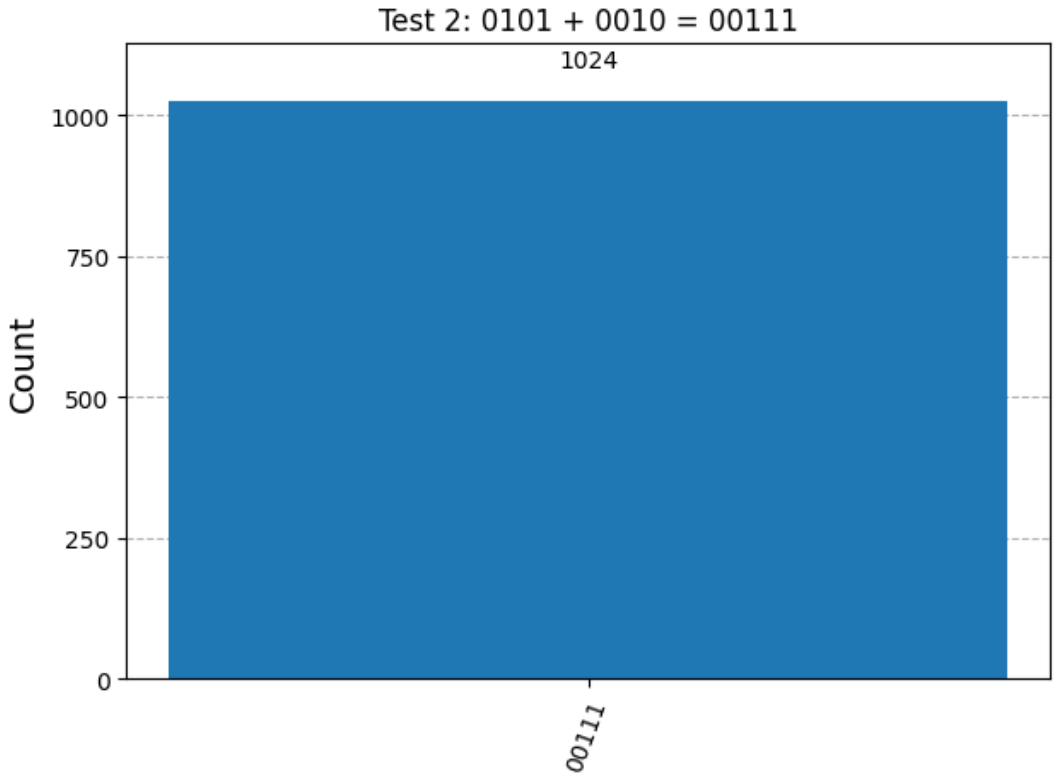
Test 2: 0101 + 0010 = 00111 (Circuit)

```
circuits[1].draw(output="mpl", fold=20)
```



Test 2: 0101 + 0010 = 00111

```
a, b = test_cases[1]
counts = all_counts[1]
title = f"Test 2: {a:04b} + {b:04b} = {a + b:05b}"
plot_histogram(counts, title=title)
```

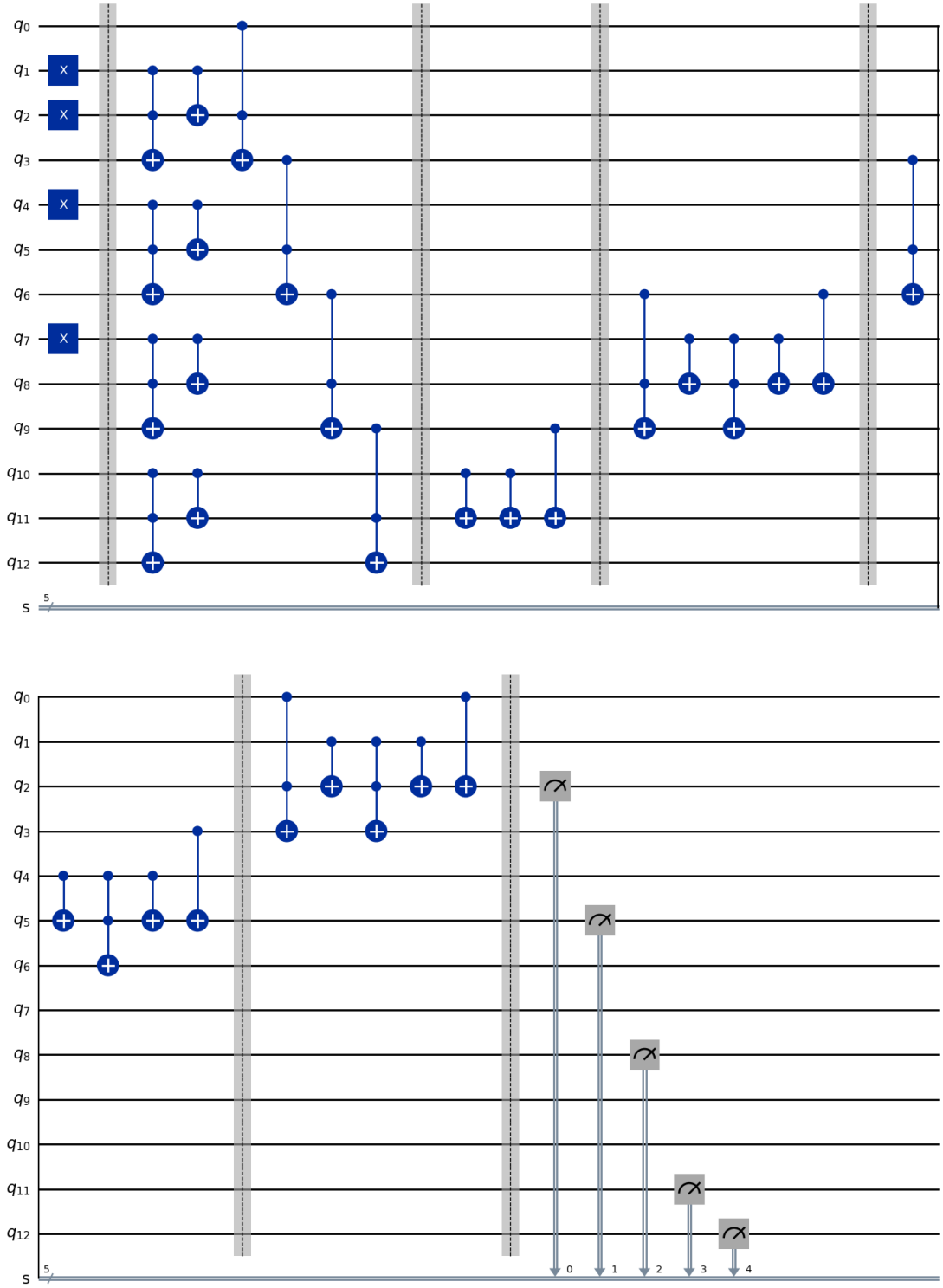


Observations

- The output 00111 occurred in all 1,024 shots, matching 0101 + 0010 = 00111 (5 + 2 = 7).
- No carries are generated for these operands; the final carry-out is zero.

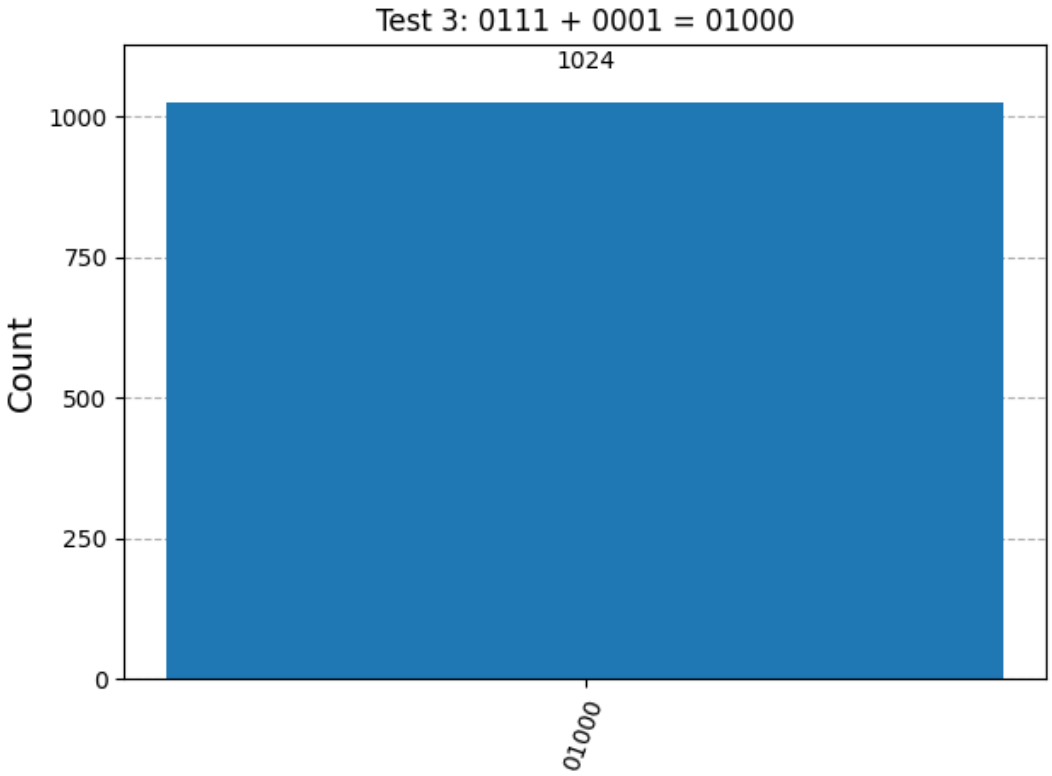
Test 3: 0111 + 0001 = 01000 (Circuit)

```
circuits[2].draw(output="mpl", fold=20)
```



Test 3: 0111 + 0001 = 01000

```
a, b = test_cases[2]
counts = all_counts[2]
title = f"Test 3: {a:04b} + {b:04b} = {a + b:05b}"
plot_histogram(counts, title=title)
```

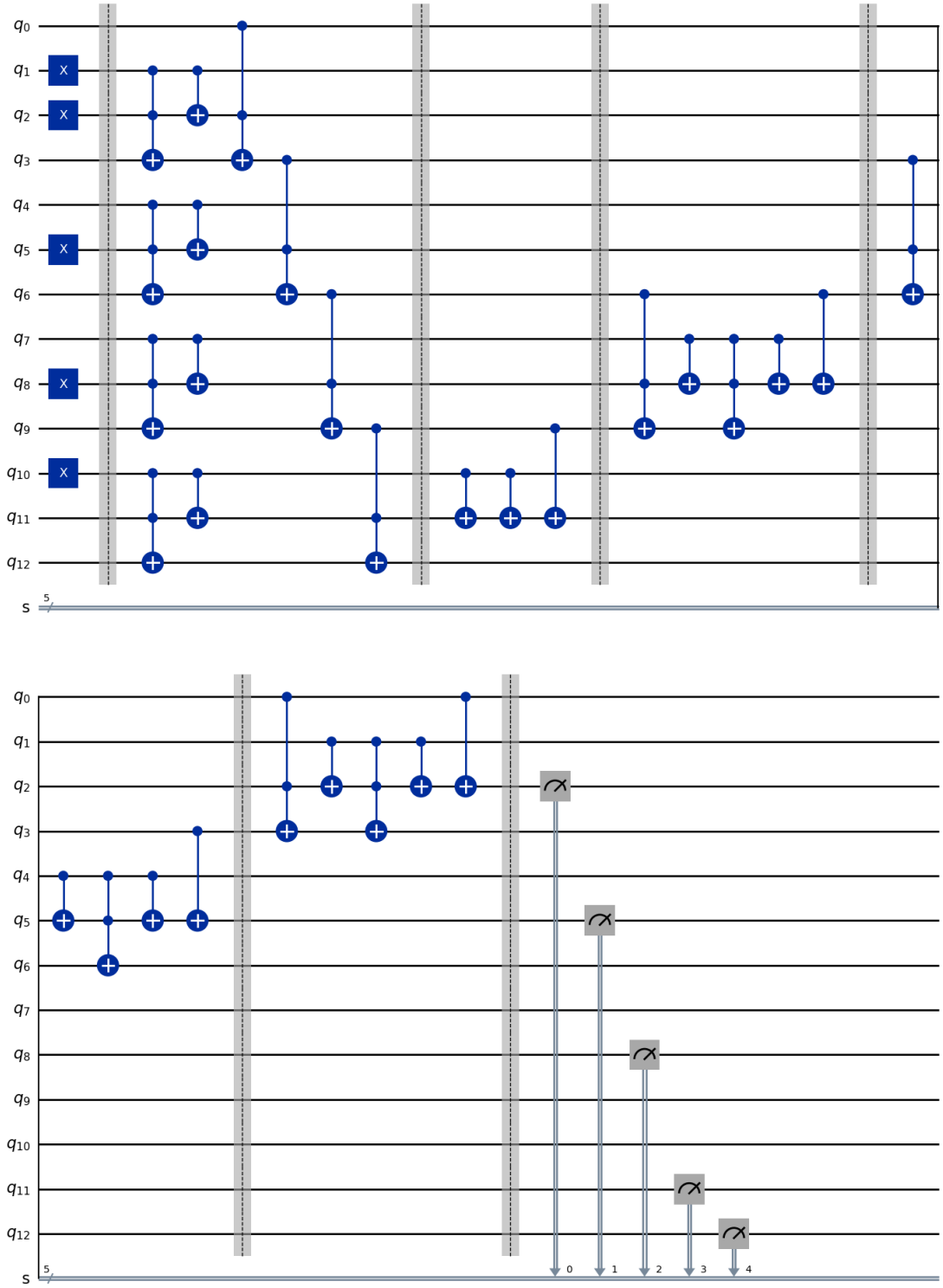


Observations

- The output 01000 occurred in all 1,024 shots, matching 0111 + 0001 = 01000 (7 + 1 = 8).
- The carry propagates through the lower three bit positions. The final carry-out is zero.

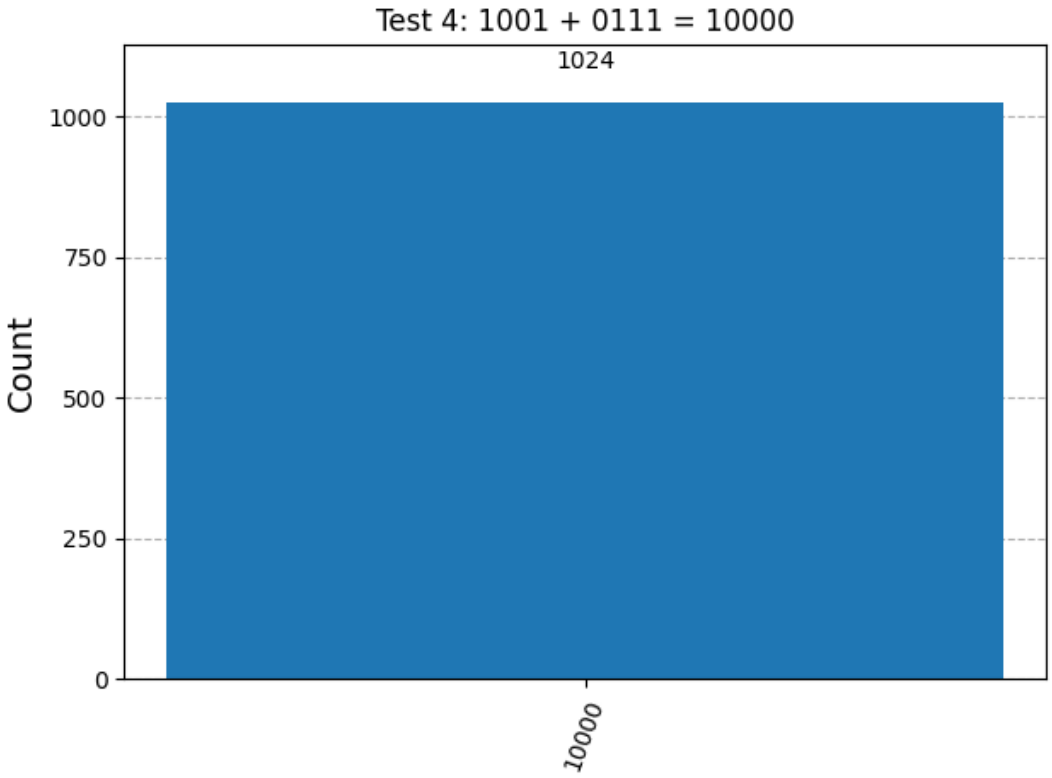
Test 4: 1001 + 0111 = 10000 (Circuit)

```
circuits[3].draw(output="mpl", fold=20)
```



Test 4: 1001 + 0111 = 10000

```
a, b = test_cases[3]
counts = all_counts[3]
title = f"Test 4: {a:04b} + {b:04b} = {a + b:05b}"
plot_histogram(counts, title=title)
```

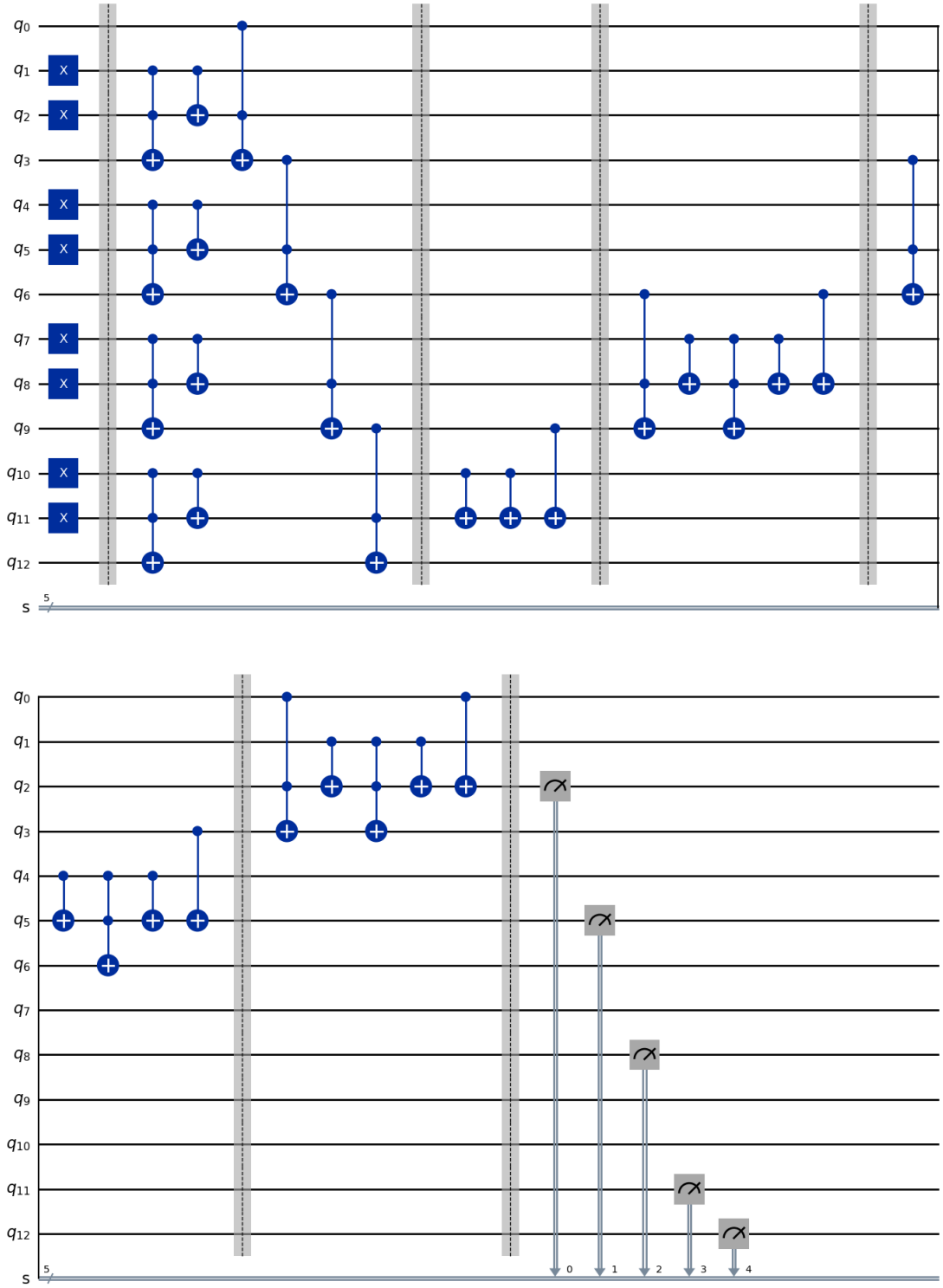


Observations

- The output 10000 occurred in all 1,024 shots, matching 1001 + 0111 = 10000 (9 + 7 = 16).
- The final carry-out is one and the lower four sum bits are 0000.

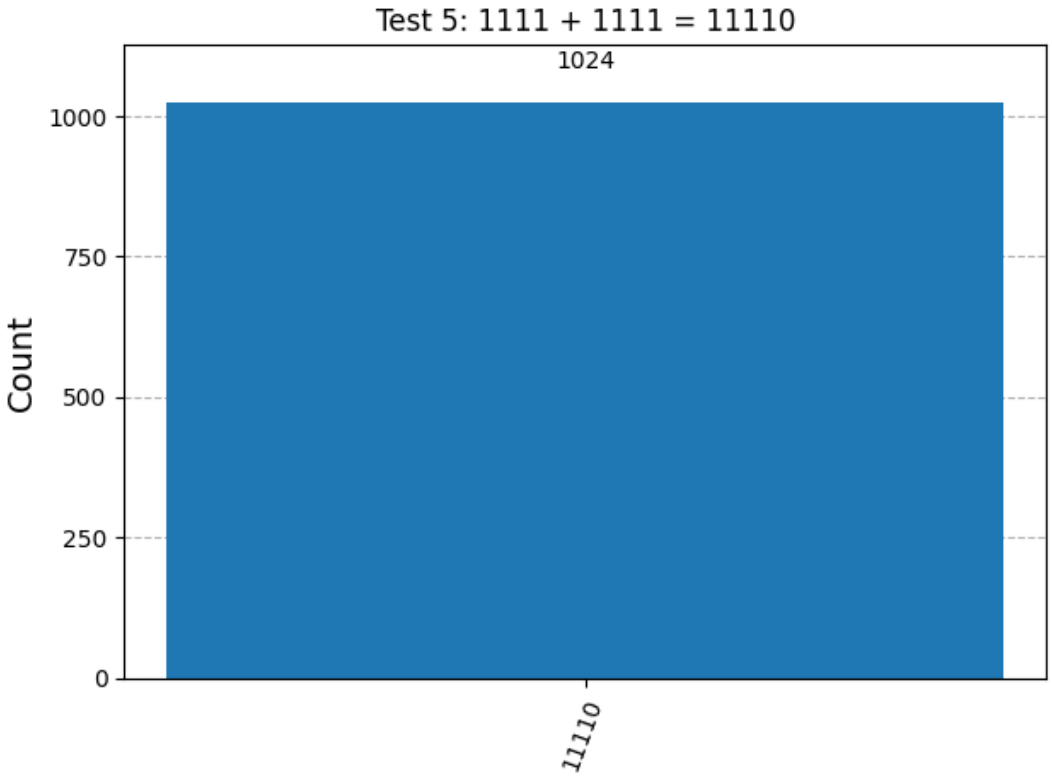
Test 5: 1111 + 1111 = 11110 (Circuit)

```
circuits[4].draw(output="mpl", fold=20)
```



Test 5: 1111 + 1111 = 11110

```
a, b = test_cases[4]
counts = all_counts[4]
title = f"Test 5: {a:04b} + {b:04b} = {a + b:05b}"
plot_histogram(counts, title=title)
```



Observations

- The output 11110 occurred in all 1,024 shots, matching 1111 + 1111 = 11110 (15 + 15 = 30).
- For the largest four-bit operands, the final carry-out is one and the lower sum bits are 1110.